

PORTFOLIO

끊임없이 새로워지는 기술과 환경,그 안에서 빠르게 적응하고 성과를 내는 유연한 김세헌입니다

KIM, SEHEON

#BACKEND

kshandy0221@gmail.com
010-3738-5871



김세헌

Backend

생년월일 : 2001. 02. 21

주소 : 서울 강서구 양천로 62길 41

이메일 : kshandy0221@gmail.com

전화번호 : 010-3738-5871

GitHub: <https://github.com/NewOld21>

스킬

- Backend: Java, Spring, TypeScript, Node.js, Nest.js
 - Frontend: React, HTML, CSS
 - Database: PostgreSQL, MySQL
 - Tooling: GitHub, Slack, Notion
-

프로젝트 - Popcorn

- 팝업스토어 예약과 굿즈 구매를 하나의 흐름으로 처리하는 MSA 기반 실시간 주문·운영 플랫폼
 - 티켓팅처럼 피크 시간대에 트래픽이 집중되는 환경을 고려하여 이벤트 기반 아키텍처와 캐시 전략을 활용한 고트래픽 처리 구조
-

학력

2020. 03 ~ 2025. 02 성결대학교 컴퓨터공학과 졸업

2017. 03 ~ 2020. 02 마포고등학교 졸업

수상 및 자격

정보처리기사 합격 (2025.12)

SQLD 합격(2025.12)

국가우수이공계 장학생 (2023.05)

프로젝트 - WaveFlow

- 음악 제작자들이 파일 버전 관리와 피드백을 효율적으로 진행할 수 있도록 설계한 실시간 협업 플랫폼
 - WebSocket 기반 실시간 통신을 활용하여 작업 변경 사항을 즉시 공유할 수 있도록 했으며, 음악 제작 과정에 맞춘 버전 관리 중심 협업 워크플로우 시스템
-

프로젝트 : Popcorn

개요

- 팝업스토어 예약과 굿즈 구매를 하나의 흐름으로 처리하는 MSA 기반 실시간 주문·운영 플랫폼
- 티켓팅처럼 피크 시간대에 트래픽이 집중되는 환경을 고려하여, 이벤트 기반 아키텍처와 비동기 메시징 구조를 적용해 대규모 동시 요청에서도 안정적으로 예약·결제를 처리

프로젝트 정보

- 기간 : 2026.01 ~ 02 (2개월)
- 팀원 : 4명
- GitHub : <https://github.com/ProfectProject>

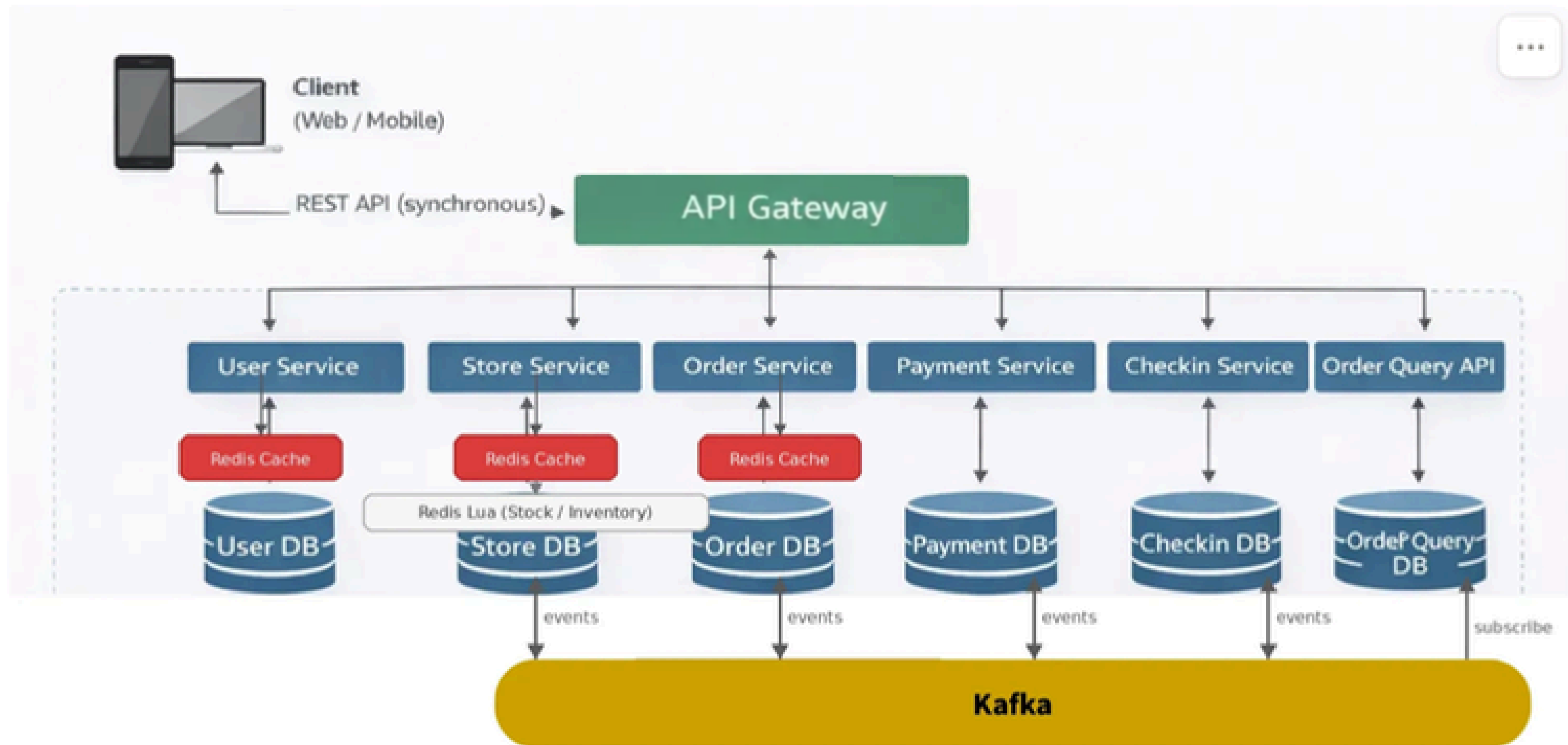
시스템 설계

- Order, Store, Payment, CheckIn, OrderQuery 등을 분리한 MSA 구조
- 서비스 간 동기 호출 의존도를 줄이기 위해 Kafka 기반 이벤트 드리븐 아키텍처 적용
- Fluent Bit, OTel, LGTM을 활용해 로그/메트릭/트레이스 통합 관측 환경 구축
- Kubernetes, Argo, Terraform, Karpenter 기반으로 배포 자동화 및 확장 가능한 운영 환경 구성

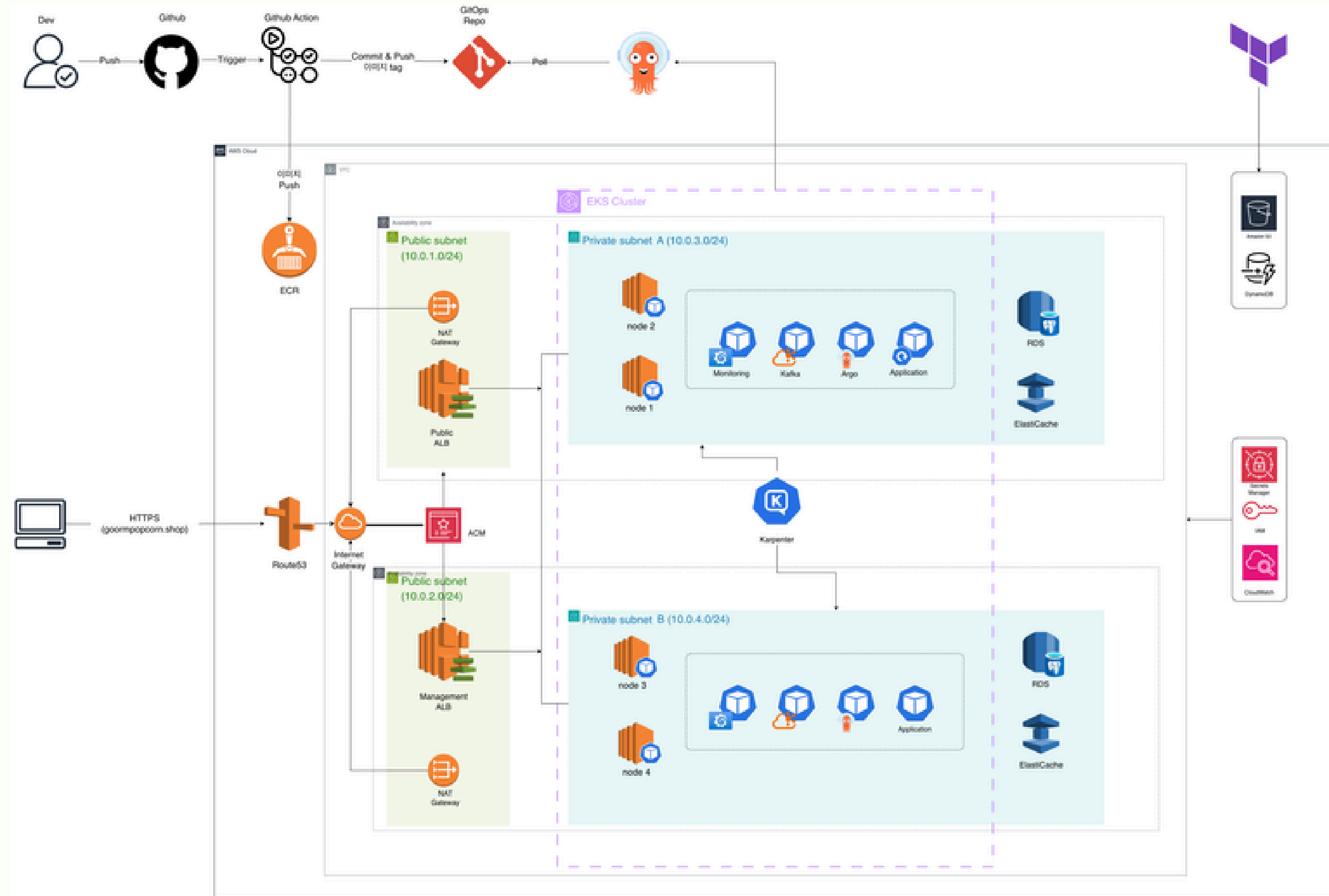
기술 스택

- Backend : Java, Kotlin, Spring Boot, PostgreSQL, Redis
- Messaging : Kafka, Kafka Connect, Debezium
- Infra : Docker, AWS, Kubernetes, Terraform, Karpenter
- DevOps & Monitoring : Argo, Loki, Tempo, Mimir, Fluent Bit, OTel, Grafana

시스템 구성도



시스템 아키텍처



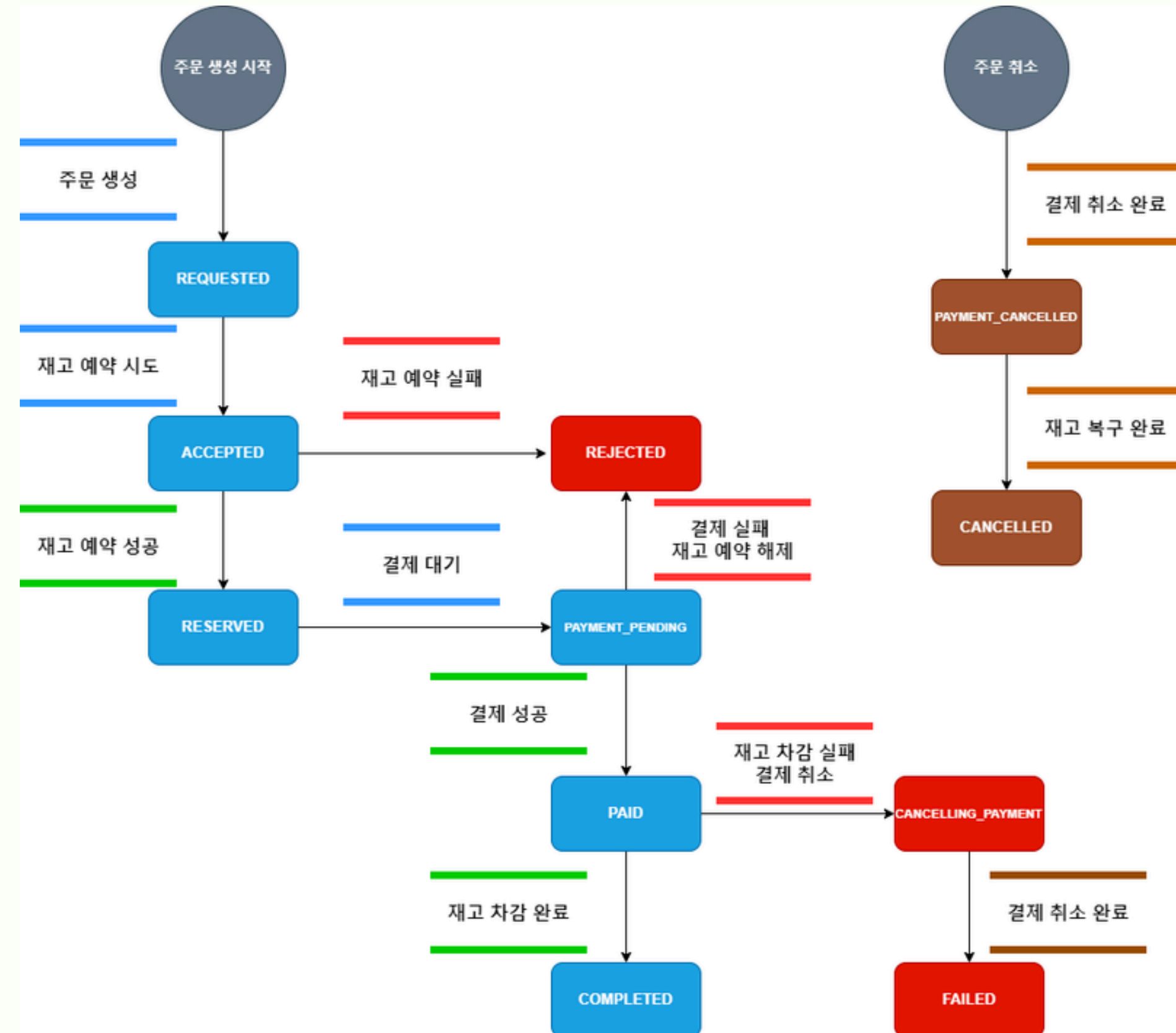
담당 역할 및 기여

Saga 오케스트레이션 기반 분산 트랜잭션 흐름 구현

- 주문, 재고, 결제, 체크인처럼 여러 서비스가 하나의 비즈니스 흐름에 참여하는 구조에서 HTTP 기반 동기 호출 대신 Kafka 이벤트 기반 Saga 오케스트레이션을 적용
- Order 서비스가 오케스트레이터 역할을 수행하고, Store, Payment, CheckIn 서비스는 이벤트를 받아 작업을 수행한 뒤 결과 이벤트를 발행하도록 설계
- 실패 시에는 Order가 보상 트랜잭션을 지휘하도록 하여 최종적 일관성을 유지

구현 내용 :

- Order 서비스 중심의 Saga 오케스트레이션 구조 설계
- 재고 예약, 결제 승인, 재고 차감, 스케줄 확정 흐름을 상태 전이로 모델링
- 결제 이후 중간 실패를 별도 상태로 관리하여 보상 경로 분리
- 참여 서비스 간 직접 호출 없이 이벤트 발행/구독 기반으로 통신
- 서비스 장애 시에도 전체 트랜잭션이 즉시 중단되지 않도록 비동기 복원력 확보



담당 역할 및 기여

중복 허용 + 멍등 전략 기반 이벤트 처리 안정성 강화

- 분산 환경에서는 중복을 허용하되 안전하게 무시할 수 있는 멍등 설계가 더 현실적이라고 판단하여 eventId 기반 멍등 처리를 전제로 설계

구현 내용 :

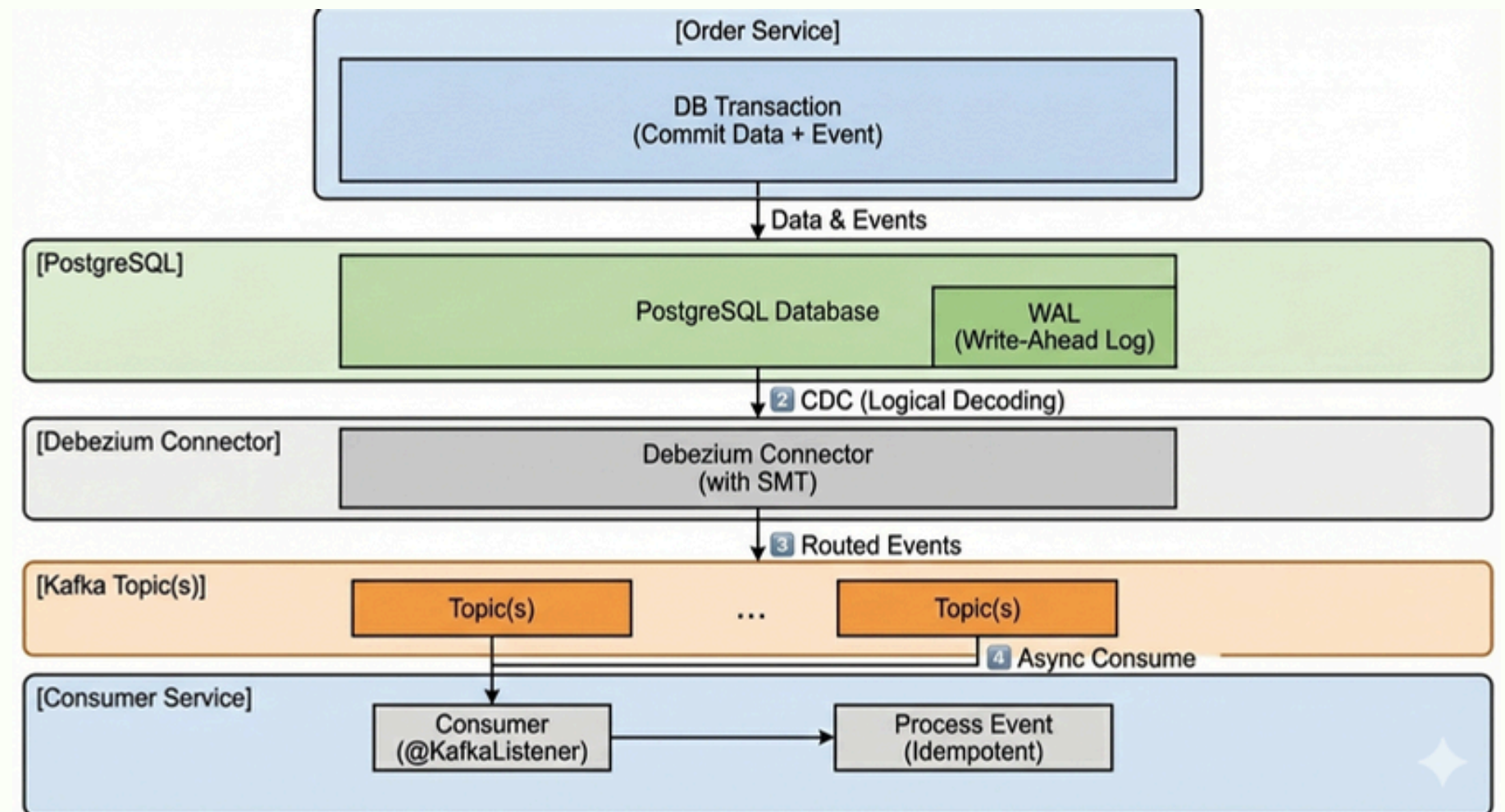
- eventId 기반 processed log / unique 제약을 통한 중복 처리 방지
- 상태 전이가 단방향으로만 진행되도록 설계
- 동일 이벤트가 재수신되더라도 상태가 뒤로 가지 않도록 가드 로직 반영

Outbox + CDC 기반 안전한 이벤트 발행 구조 구현

주문 생성, 결제 승인, 재고 차감과 같이 유실되면 안 되는 핵심 이벤트에 대해 비즈니스 데이터 저장과 이벤트 기록을 하나의 트랜잭션으로 묶는 Outbox 패턴을 도입

구현 내용 :

- 서비스별 outbox_events 테이블 설계 및 이벤트 스키마 표준화
- event_id UNIQUE` 제약을 통한 멍등성 및 중복 방지 구조 설계
- Debezium PostgreSQL Source Connector + Outbox Event Router(SMT) 적용
- Polling 방식 대신 CDC 기반으로 변경하여 이벤트 지연 약 0.5초 감소와 DB 부하 감소



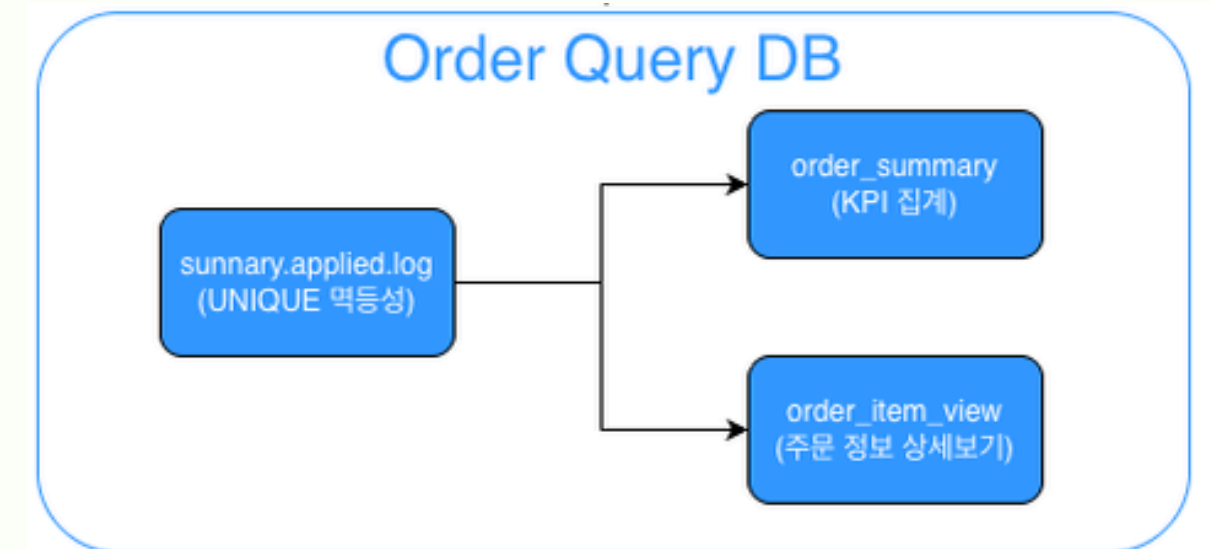
담당 역할 및 기여

CQRS 조회 모델(OrderQuery) 설계 및 이벤트 기반 집계 처리

- 운영 화면과 조회 성능을 분리하기 위해 OrderQuery 서비스를 별도로 두고, Order, Payment, Store, CheckIn 이벤트를 구독하여 조회 전용 모델을 구성

구현 내용 :

- 집계 테이블과 summary_applied_log 기반 멍등 카운터 반영
- 주문 생성 / 결제 승인 / 체크인 완료 등 주요 이벤트를 조회 모델에 반영
- 이벤트 중복 수신 시에도 집계값이 깨지지 않도록 applied log 기반 가드 적용



운영 가시성을 위한 구조화 로그 파이프라인 구축

- MSA 환경에서는 장애가 발생했을 때 서비스 간 흐름을 빠르게 추적할 수 있어야 한다고 판단해, JSON 구조화 로그를 표준화하고 Fluent Bit, Loki, Grafana 기반 로그 수집/조회 파이프라인을 구축

구현 내용 :

- Logback JSON 포맷 표준화
- Fluent Bit로 컨테이너 로그 수집 및 JSON 파싱
- Loki 적재 후 Grafana에서 서비스 / traceId 기준 조회 가능하도록 구성
- 요청 단위 추적과 에러율 확인이 가능한 운영 로그 구조 마련



트러블 슈팅

Redis Lua 기반 재고 HOLD 구조 설계로 동시성 문제 해결

상황

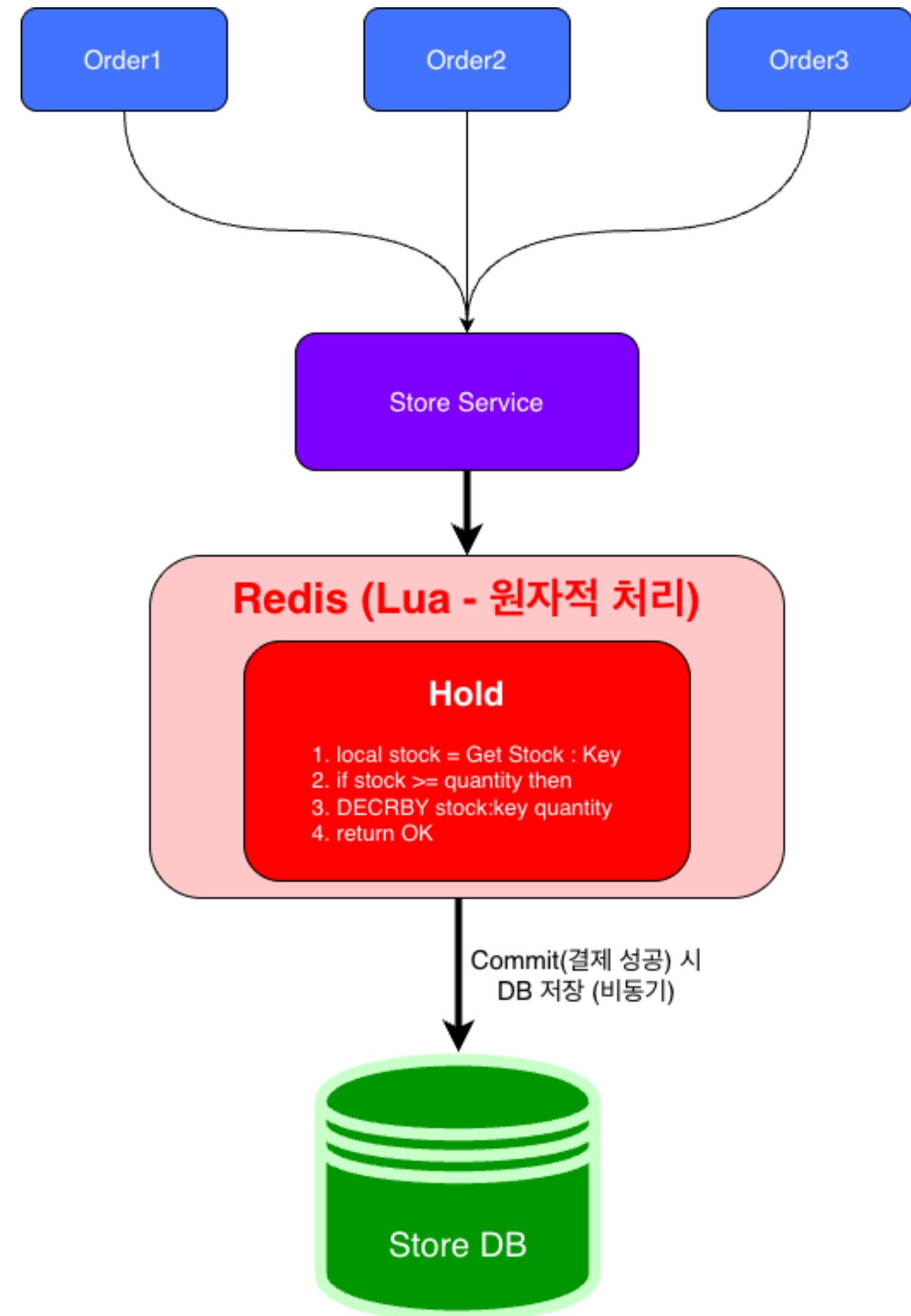
- 팝업 예약과 굿즈 구매는 짧은 시간에 동시 요청이 집중
- 단순 DB 차감 방식으로는 마지막 재고를 여러 사용자가 동시에 점유하거나 재고가 음수로 내려가는 문제가 발생

해결

- Store 서비스가 주문 생성 이벤트를 수신하면 DB가 아니라 Redis에서 먼저 재고를 처리하도록 구조를 변경
- Redis Lua 스크립트로 재고 확인과 차감(HOLD)을 원자적으로 실행해, 동시 요청 상황에서도 정확한 선점이 가능하도록 설계
- 또한 결제 전에는 Redis에서만 HOLD를 유지하고, 결제 성공(COMMIT) 시점에만 DB 재고에 최종 반영하도록 분리
- 결제 실패나 만료 시에는 Redis에서 RELEASE 처리하고 DB는 최종 결과 기록만 담당하도록 설계

결과

- 동시 요청 상황에서도 재고가 음수로 내려가지 않도록 보장
- 락 경합 없이 빠른 재고 선점 처리 가능
- 스케줄 + 굿즈 복합 요청에서도 부분 차감 없이 일관된 처리 보장
- DB는 확정 재고만 관리하고 Redis는 실시간 경쟁 처리에 집중하도록 역할 분리



프로젝트 : WaveFlow

개요

- 멀티트랙 기반 실시간 음악 협업 플랫폼
- 여러 음악 제작자가 동시에 작업하고 피드백을 주고받을 수 있도록 설계한 협업 플랫폼
- 음악 제작 과정에서 발생하는 버전 관리 문제와 협업 비효율을 해결하기 위해 Track → Version → Stem 구조 기반의 버전 관리 시스템과 리뷰 워크플로우를 구현

프로젝트 정보

- 기간 : 2025.07 (1개월)
- 팀원 : 5명
- GitHub : <https://github.com/Team-Honey-Badgers>

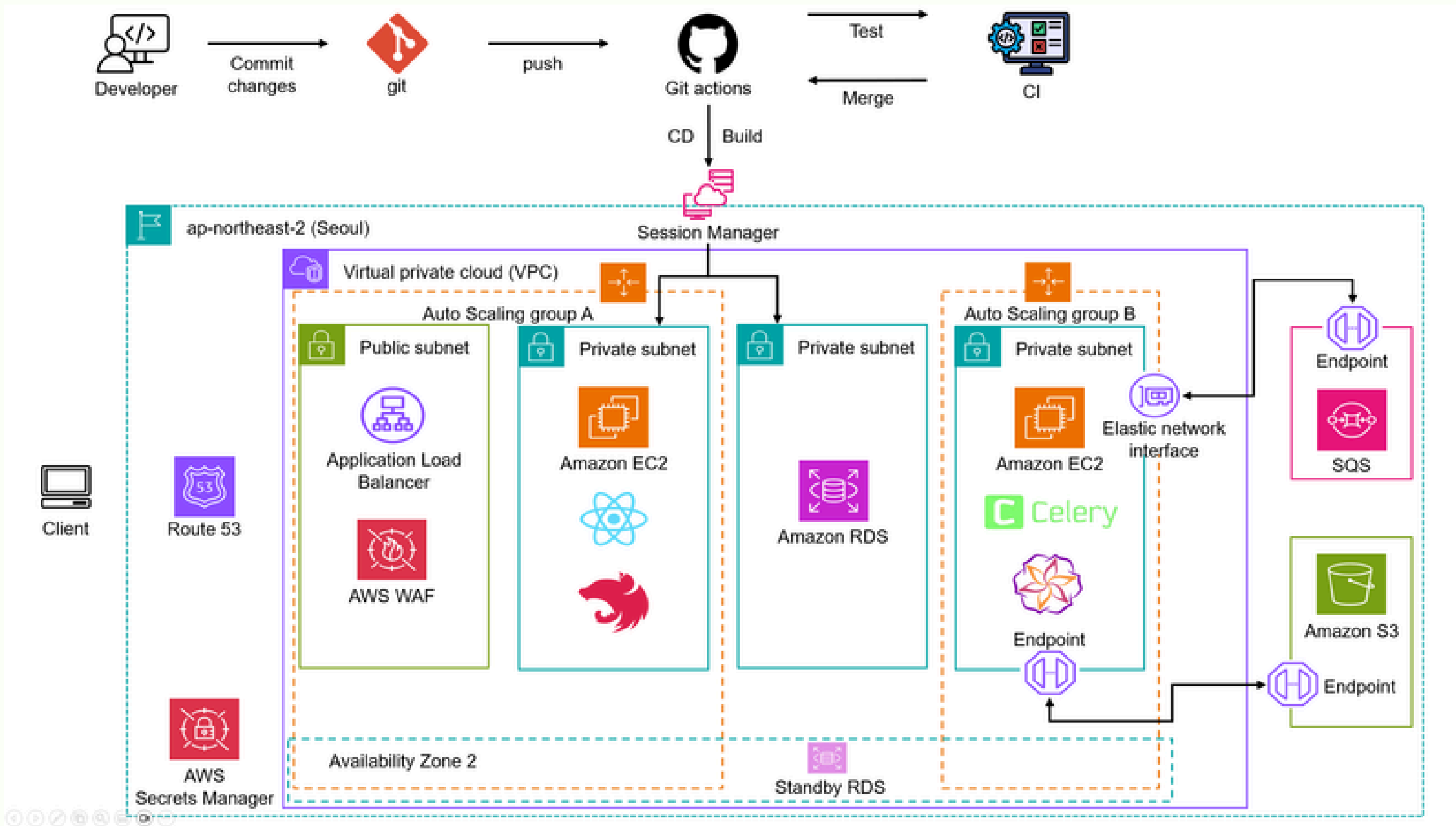
시스템 설계

- NestJS 기반 RESTful API 서버 구축
- AWS S3 Presigned URL을 활용한 대용량 오디오 파일 업로드 구조 설계
- WebSocket 기반 실시간 협업 환경 구축
- Track / Version / Stem 기반 버전 관리 데이터 모델 설계

기술 스택

- Backend : NestJS, PostgreSQL, TypeScript
- Frontend : React, TailwindCSS
- Infra : AWS S3, EC2

시스템 아키텍처



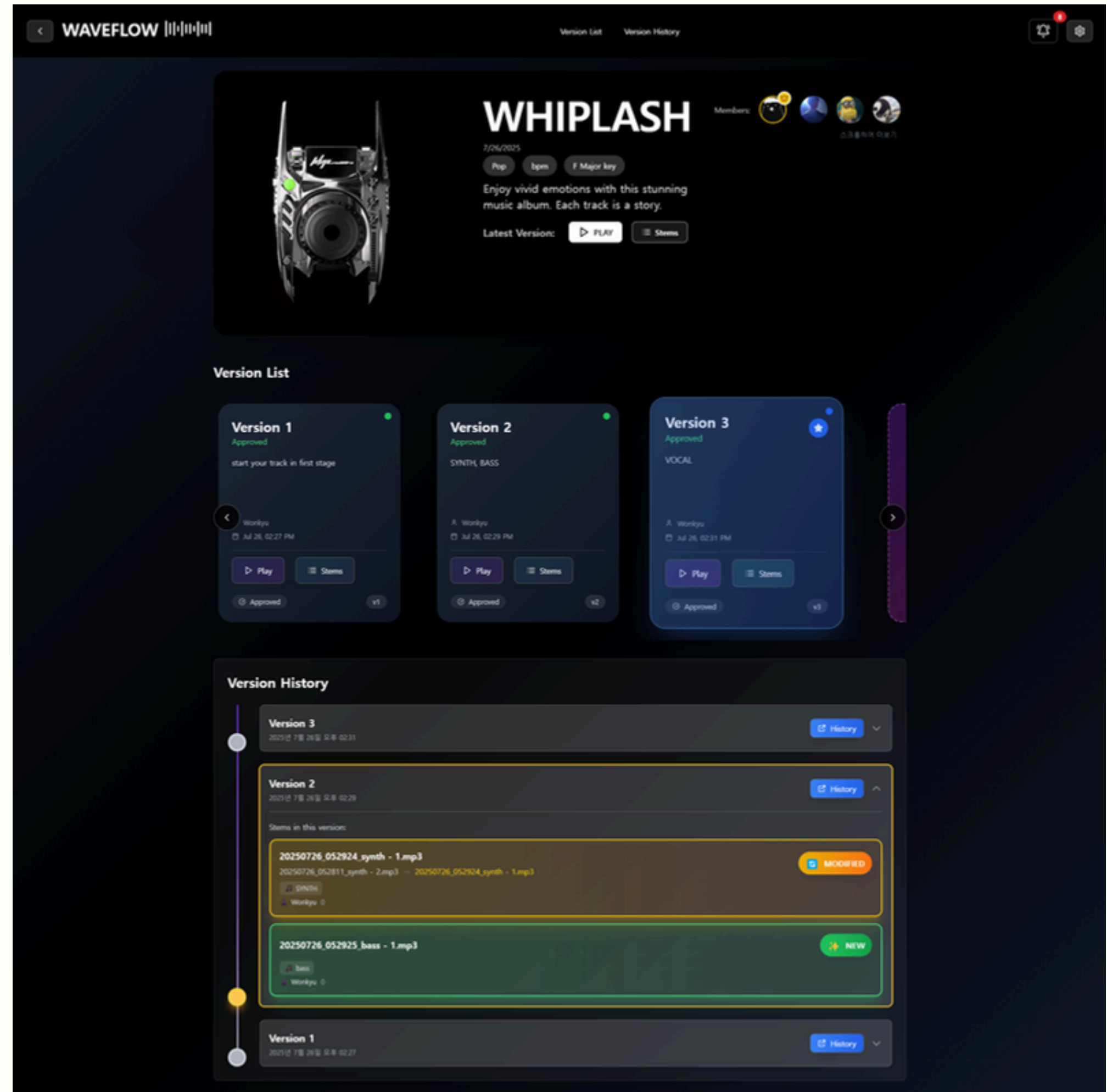
담당 역할 및 기여

버전 히스토리 관리 UI 구현

- 음악 제작 과정에서 발생하는 버전 관리 문제를 해결하기 위해 Track, Version 구조의 버전 히스토리 UI 설계
- 이전 작업 상태를 쉽게 확인하고 복원할 수 있도록 버전 관리 인터페이스 개선

구현 내용 :

- 버전 히스토리를 타임라인 형태로 시각화
- 각 버전별 변경 정보 및 Stem 파일 정보 표시
- 원클릭 이전 버전 롤백 인터페이스 구현



담당 역할 및 기여

버전 관리 데이터 모델 설계 및 구현

음악 제작 과정에서 발생하는 버전 관리 문제를 해결하기 위해
Track → Version → Stem 구조의 관계형 데이터 모델을 설계

구현 내용 :

- Track / Version / Stem 관계형 스키마 설계
- 트랙 수정 시 자동 버전 생성 로직 구현
- 이전 버전으로 롤백 가능한 구조 설계

인증 및 권한 관리와 실시간 협업 기능 구현

JWT 기반 Stateless 인증 및 다수의 제작자가 동시에 작업할 수 있도록 WebSocket 기반 실시간 협업 환경을 구현

구현 내용 :

- WebSocket 기반 실시간 이벤트 처리
- JWT 토큰 기반 협업자 권한 관리 및 WebSocket 인증
- 협업 상태 및 리뷰 상태 실시간 업데이트

대용량 오디오 파일 업로드 시스템 구현

음악 협업 서비스 특성상 대용량 오디오 파일 업로드가 빈번하게 발생하기 때문에
AWS S3 Presigned URL 기반 파일 업로드 구조를 설계

구현 내용 :

- NestJS 기반 Presigned URL 발급 API 구현
- 클라이언트에서 S3로 직접 업로드하도록 구조 설계
- 서버 부하 없이 청크 단위로 나눠 병렬 대용량 파일 업로드 처리

트랙 관리 및 버전 관리 API 구현

트랙 관리와 버전 관리 기능을 위한 REST API를 설계하고 구현

구현 내용 :

- 30개 이상의 REST API 엔드포인트 구현
- 트랙 조회 / 버전 관리 / 롤백 기능 구현
- 파일 업로드 및 리뷰 기능 API 구현

담당 역할 및 기여

실시간 협업 알림 시스템 구현 (WebSocket 기반)

- 협업 환경에서 트랙 변경 및 리뷰 상태를 사용자에게 즉시 전달하기 위해 WebSocket 기반 실시간 알림 시스템 구현
- API polling 방식 대신 이벤트 기반 상태 업데이트 구조로 개선하여 불필요한 네트워크 요청 감소

구현 내용 :

- JWT 기반 인증 후 WebSocket 연결 처리
 - 트랙 변경 / 리뷰 상태 변경 이벤트 알림 수신
 - 알림 발생 시 UI 상태 즉시 갱신
-

대용량 오디오 업로드 성능 개선 (Presigned URL 기반 업로드)

- 대용량 오디오 파일 업로드 시 서버 부하를 줄이기 위해 AWS S3 Presigned URL 기반 업로드 구조 설계
- 대용량 오디오 업로드 시 서버를 거치지 않도록 구조 개선으로 업로드 과정에서 발생하는 서버 부하 감소

구현 내용 :

- Presigned URL 발급 후 클라이언트 → S3 직접 업로드 구조 구현
 - 업로드 진행 상태(progress) UI 구현
 - 업로드 실패 시 재시도 로직 구현
-

트러블 슈팅

증분 스냅샷 도입

상황

- 초기에는 Stage마다 전체 Stem 파일을 스냅샷 형태로 복제 저장
- 동일한 Stem 파일이 중복 보관되며 스토리지 낭비가 발생
- 롤백 시 전체 스냅샷을 일괄 복원해야 했기에 복구 시간 증가

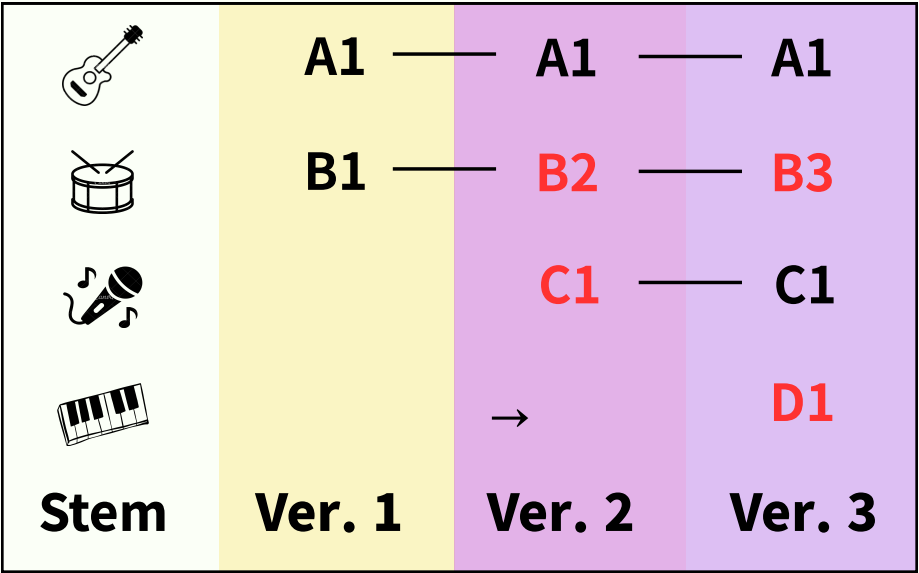
해결

Stage마다 수정되거나 추가된 Stem만 별도로 저장하는 증분 스냅샷 구조 도입

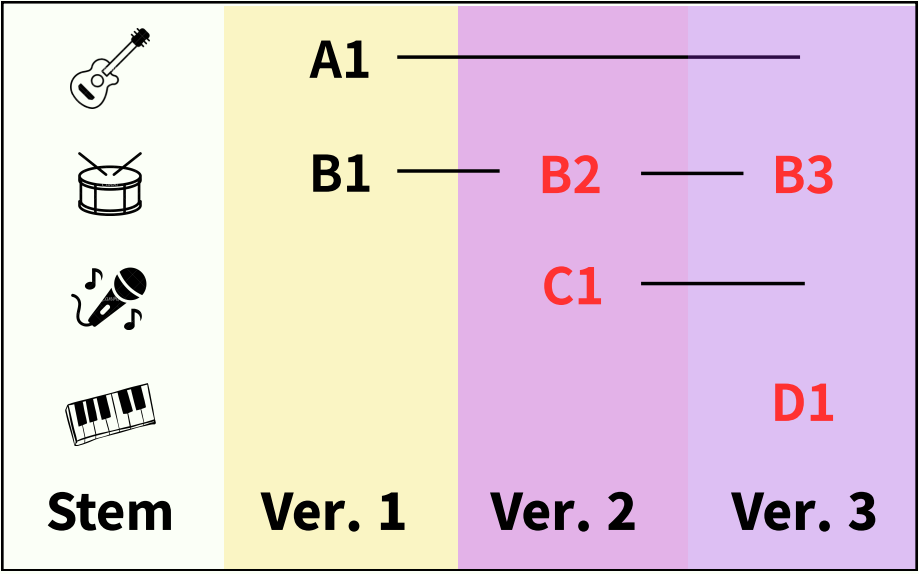
- 변경 없는 파일은 이전 Stage의 레퍼런스를 참조하도록 설계
- 롤백 시 Cascade Delete를 활용해 이후 Stage만 삭제하고, 증분된 스냅샷만 복원

결과

- 스토리지 사용량 약 60% 감소
- 롤백 속도 약 2배 개선



기존



개선

CONTACT

kshandy0221@gmail.com

010-3738-5871

THANK YOU